01 / PRODUCT & PURPOSE

Continuity memory for film productions

An AI script supervisor that connects screenplay facts, persistent production records and tool-assisted continuity reasoning.

Agentic Cinema: The Blockbuster Hackathon

Submission track: ClickHouse. SceneKeeper combines a Streamlit interface, Gemini 3.6 Flash and ClickHouse Cloud, with the official mcp-clickhouse server in the audit query path.

The problem SceneKeeper addresses

Film continuity depends on small details remaining coherent across scenes: a jacket, an injury, who holds a prop, its condition and where it appears. Those facts are difficult to compare when they are scattered across screenplay text and production records.

SceneKeeper turns uploaded screenplay text into scene-linked records, then lets an agent retrieve and compare the relevant evidence. It is a text-and-records workflow for script-supervisor review; the current code does not analyze filmed footage or images.

Current end-to-end workflow

01 Import

Upload a UTF-8 TXT screenplay in Streamlit. Gemini extracts scenes, character states and props.

02 Remember

The importer assigns a production_id, writes records directly to ClickHouse Cloud and makes the new production active.

03 Audit

A production-aware Gemini request generates SQL. An application guardrail checks scope before queries go through official mcp-clickhouse.

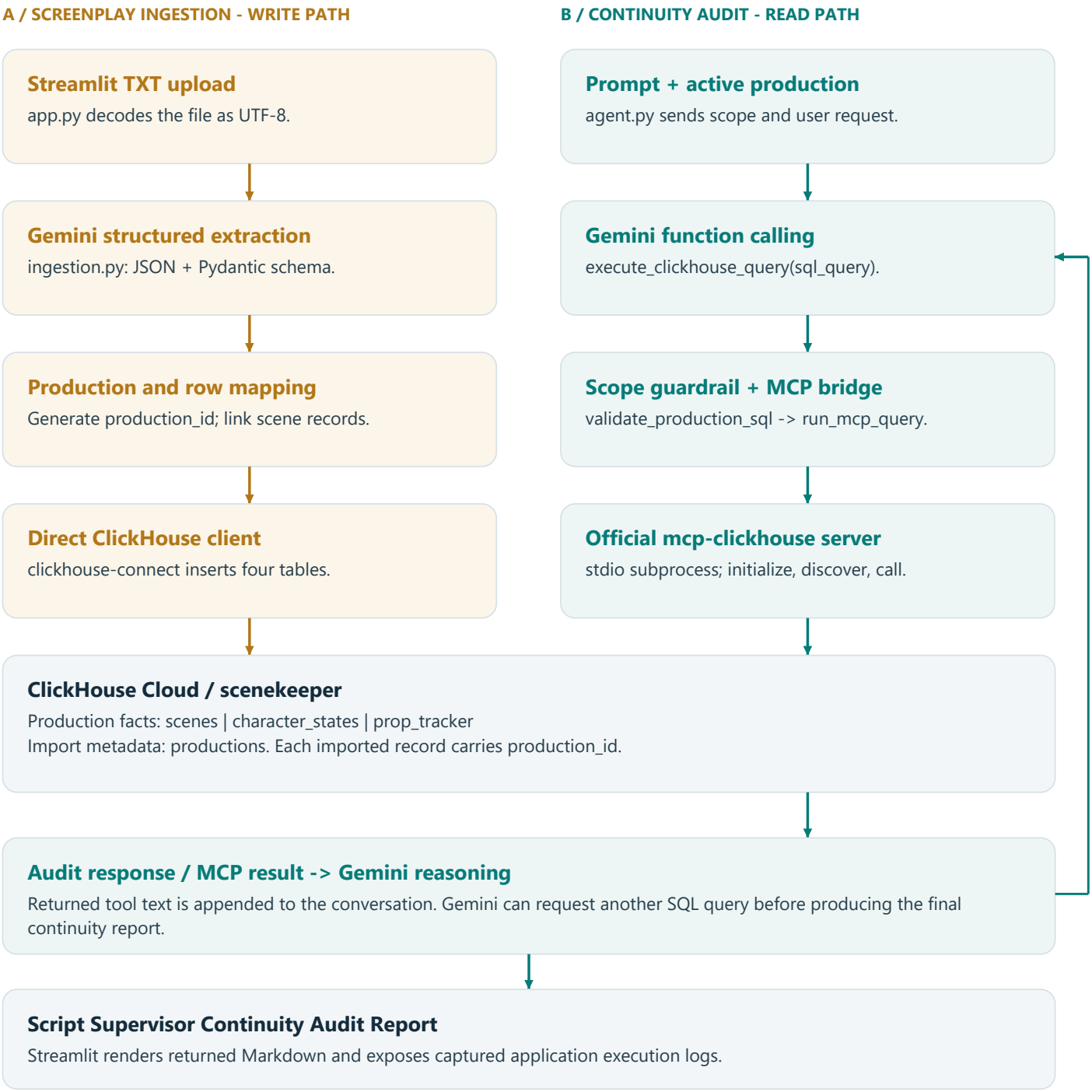
04 Explain

Query results return to Gemini for further tool calls or a continuity report, which Streamlit displays with captured execution logs.

02 / SYSTEM ARCHITECTURE

Two paths. One production memory.

Ingestion writes directly. Continuity audits use the official MCP runtime. The diagram describes code paths, not a fabricated application screenshot.



Path boundary: ingestion also reads current maximum IDs through the direct client. The audit read path uses MCP; it is not a database-level read-only or tenant-security guarantee.

Code basis: ingestion.py:45-64,70-225; agent.py:47-59,98-189; mcp_client.py:9-61

Implementation snapshot: 09 Sep 2026 | Local source review; no live systems changed

03 / INGESTION & STORAGE

From screenplay text to records

Dynamic ingestion uses Gemini structured output and direct ClickHouse inserts. It does not use MCP to write the screenplay data.

Schema-led extraction

ingest_txt_screenplay() sends the entire text to **gemini-3.6-flash** with temperature 0.1, JSON output and the **ScreenplayExtraction** Pydantic schema. The instruction says to extract only explicit screenplay facts and leave missing details as empty strings. These are model instructions, not a guarantee of extraction accuracy.

Record family	Fields implemented
Scenes	scene_number, title, location, time_of_day, script_summary
Character states	scene_number, character_name, wardrobe_description, physical_condition, knowledge_state, notes
Props	scene_number, prop_name, held_by, condition, location_in_set
Production metadata	production_id, title, scene_count, character_count, prop_count; imported_at has a database default

Linking and writing the production

The importer derives a production_id from a sanitized title plus six UUID hex characters. It reads each fact table's current maximum ID and increments sequential IDs, maps extracted scene numbers to scene_id values, then inserts scenes, character states, props and production metadata.

Unmatched scene references fall back to the first inserted scene. If extraction contains no scenes, the importer creates a General Scene container. After success, app.py stores the new production_id in Streamlit session state and reruns the UI.

> **Storage and migration boundary**
>
> migrate.py adds production_id with a demo_sample default to the three existing fact tables and creates the productions metadata table with ReplacingMergeTree. It assumes the original fact tables already exist; it is not a complete initial schema bootstrap.

Implementation limits: inserts are separate operations; there is no transaction, rollback, deduplication or concurrency-safe ID allocator in the application code. The UI currently accepts TXT files only.

04 / REASONING, MCP & ISOLATION

An evidence-retrieval audit loop

Gemini decides what to retrieve. Python scopes the request. The official MCP server executes runtime queries against ClickHouse.

Gemini integration

query_agent() declares one Gemini-facing function, **execute_clickhouse_query**, with a sql_query string argument. It prefixes the user request with the active production ID and instructs every fact-table query to filter that production. Gemini 3.6 Flash runs at temperature 0.1 and can request multiple tool calls per response.

The agent preserves conversation history and returns each tool result to Gemini. It permits up to 10 turns by default, then requests a final summary with no tools. generate_with_retry() allows three attempts per call, waiting 2 and 4 seconds before retries; it catches exceptions generally, rather than classifying quota or network failures.

Official mcp-clickhouse integration

mcp_client.py launches the **mcp-clickhouse** executable as a stdio subprocess with inherited environment configuration. For each query it opens an MCP ClientSession, initializes the session, lists tools, selects a query-like tool and calls it. Returned content is joined into text and passed back to the agent.

The bridge chooses tools by name heuristics and attempts to inspect their query parameter schema, defaulting to query. Gemini sees the application's single query wrapper, not the complete discovered MCP tool catalog. The synchronous entry point uses asyncio.run(); the connection is opened per query, not pooled across the audit.

Production isolation: what is implemented

Application-level scope checks

New rows carry production_id. The UI starts at demo_sample and switches to the imported production. Before an audit query reaches MCP, validate_production_sql() looks for fact-table names and an exact active-production equality string in normalized SQL.

This is a textual guardrail, not SQL parsing, join-by-join validation, database row-level security or user authorization. It does not establish secure multi-tenant isolation. Rejected queries and MCP exceptions are returned to Gemini as error text so the conversation can continue.

Audit read path means the intended retrieval workflow. SceneKeeper itself does not enforce a SELECT-only SQL policy or prove the permissions of the server's database account.

05 / EXPERIENCE & OUTPUT

From evidence to a supervisor report

The interface presents production-wide auditing, a custom-question input, production-table descriptions and a screenplay import workflow.

Interface element	Current implementation
Audit Entire Production	Uses a preset comparison prompt covering wardrobe, physical condition, knowledge, props, ownership, condition and location across the active production. Its Run Continuity Check button invokes query_agent().
Custom Prompt	The radio option and text area exist. In this snapshot the run button is nested under the other branch, so Custom Prompt has no submit action.
Live MCP Execution Logs	Successful runs expose captured stdout in an expander: user request, SQL sent through the MCP wrapper, returned results and agent messages. They appear after execution, not as a live stream.
Production Tables	Describes the three fact tables. This tab does not query or display their live rows. Sidebar status badges are static messages, not health checks.

Script Supervisor Continuity Audit Report

The agent is instructed to explain continuity issues and provide an actionable report; Streamlit renders its returned text as Markdown. **The code does not enforce the following four headings or a fixed severity schema.** This judge-facing outline describes how to organize and assess the generated report.

Executive Summary

A concise interpretation of the retrieved production evidence and overall continuity concerns.

Critical Continuity Inconsistencies

Scene-to-scene changes in wardrobe, injuries, knowledge or props, with supporting records and an explanation of why the change may matter.

Verified Consistent Elements

Stable details supported by retrieved records. The entire-production prompt explicitly asks for important consistencies.

Recommended Production Actions

Actionable follow-up for the crew, as requested by the agent instruction. Creative intent and missing context still require human review.

06 / STACK, EVIDENCE & SUBMISSION

Built with a focused Python stack

Versions below are pinned in the repository requirements.txt. They are not an independent verification of the hosted deployment environment.

Component	Repository pin / role
Streamlit	1.63.0 - upload, session state, audit UI and Markdown output
Google GenAI SDK	google-genai 2.22.0 - Gemini 3.6 Flash extraction and reasoning
ClickHouse Python client	clickhouse-connect 1.8.0 - ingestion inserts, ID reads and migration writes
Official MCP runtime	mcp-clickhouse 0.6.0; mcp 2.1.1 - subprocess server and ClientSession
Structured data / config	pydantic 2.13.5; python-dotenv 1.2.3 - extraction schema and environment loading

Language and hosting: Python application; Python version is not pinned by these source files. README documents Streamlit Community Cloud hosting. **License:** MIT, copyright 2026 Aisha Mohamed.

Submission entry points

Live application
https://scenekeeper.streamlit.app
GitHub repository
https://github.com/aishahamidm-droid/scenekeeper
Hackathon: Agentic Cinema: The Blockbuster Hackathon
Partner track: ClickHouse

Links are recorded in README and supplied for the submission. Live deployment availability was not tested during this documentation review.

Evidence and scope

Inspected: app.py, agent.py, ingestion.py, mcp_client.py, migrate.py, requirements.txt, README.md, LICENSE, .gitignore and test_screenplay.txt. The screenplay fixture is empty; no screenshots or image assets are present. The architecture figure is an original vector diagram derived from these files.

Excluded claims: video/image continuity analysis; a working Custom Prompt submit flow; enforced report headings; streamed or persistent MCP telemetry; database-enforced tenant security; transactional or concurrency-safe ingestion; measured accuracy, latency or scale. The implementation does not establish these capabilities.

This document reflects the local implementation on 09 Sep 2026. No application files, database data or configuration were changed for this review. No secrets or environment-file contents are included.